Big Data Assignment 2 Report

n.zagainov@innopolis.university
GITHUB: https://github.com/MatPlotLib-Guild/big-data-assignment2

Methodology

Methodology

The search engine was implemented as a four-stage pipeline: service startup, data preparation, offline indexing, and online retrieval. HDFS is used to store both the prepared corpus and the index artifacts, Hadoop Streaming MapReduce is used to build the inverted index, Cassandra is used as the serving database, and Spark is used for data preparation and BM25 ranking. The main execution script is app.sh, which starts the distributed services, creates a Python virtual environment, installs the required dependencies, prepares the data, builds and stores the index, and finally runs a sample search query. The script start-services.sh starts HDFS, YARN, and the MapReduce history server, and also uploads a compact Spark runtime archive to HDFS so that Spark-on-YARN submissions are more stable.

The data preparation stage is implemented by prepare_data.py and prepare_data.sh. In its default mode, prepare_data.py reads one parquet file with Spark, selects the id, title, and text fields, samples a fixed number of rows (100 in the current configuration), sanitizes file names, and writes each article as a plain UTF-8 text file named <doc_id>_<doc_title>.txt. Then prepare_data.sh uploads the parquet file to HDFS, runs prepare_data.py to generate the local text documents, stores these raw documents in HDFS under /data, and invokes prepare_data.py in a second mode to build the final MapReduce input in /input/data. In this second mode, Spark reads all files from /data using RDD wholeTextFiles, extracts the document id and title from the file name, normalizes the text into a single line, and writes the prepared dataset to /input/data as one partition. Therefore, the input to the indexer is a single-partition HDFS dataset where each line has the format <doc_id>\t<doc_title>\t<doc_text>.

The indexing stage is controlled by create_index.sh, index.sh, mapper1.py, reducer1.py, mapper2.py, reducer2.py, and split_index_output.py. index.sh is a wrapper that runs create_index.sh and then store_index.sh. create_index.sh executes two Hadoop Streaming pipelines. The first pipeline creates postings and document statistics. The second pipeline creates the vocabulary with document frequencies.

mapper1.py processes one prepared input record at a time. It reads a line from standard input, splits it into doc_id, title, and text, tokenizes the text, counts term frequencies inside the document, and counts the total number of tokens. Its output has two record types. For each distinct term it emits POSTING term doc_id title term_frequency. At the end of the document it emits DOC __meta__ doc_id title doc_length. reducer1.py aggregates records with the same key. For POSTING records it outputs POSTING term doc_id title total_term_frequency. For DOC

records it outputs DOC doc_id title doc_length. While processing all DOC records, it also counts the number of indexed documents and sums their lengths, and finally emits one corpus-level row CORPUS document_count total_doc_length. Thus, pipeline 1 produces the raw material for the inverted index, the document lengths, and the corpus-wide statistics needed later for BM25.

The mixed output of pipeline 1 is first stored in /tmp/indexer/pipeline1. Then split_index_output.py separates it into three local TSV files according to the record type: postings.tsv, doc_stats.tsv, and corpus_stats.tsv. These files are uploaded to the final HDFS index locations. /indexer/index stores the postings list, where each row represents one term in one document together with its term frequency. /indexer/doc_stats stores per-document metadata, specifically document id, title, and document length. /indexer/corpus_stats stores corpus-level values, namely the number of indexed documents and the total document length, from which average document length is derived. These three artifacts are required by BM25.

The second MapReduce pipeline computes the vocabulary. mapper2.py reads the output of pipeline 1 and ignores everything except POSTING rows. For each posting it emits VOCAB term 1. Since each posting already corresponds to one term-document pair, counting these values gives the number of documents containing the term. reducer2.py sums the counts for each term and outputs VOCAB term document_frequency. The result is stored in /indexer/vocabulary. This artifact provides the document frequency needed for the IDF component of BM25.

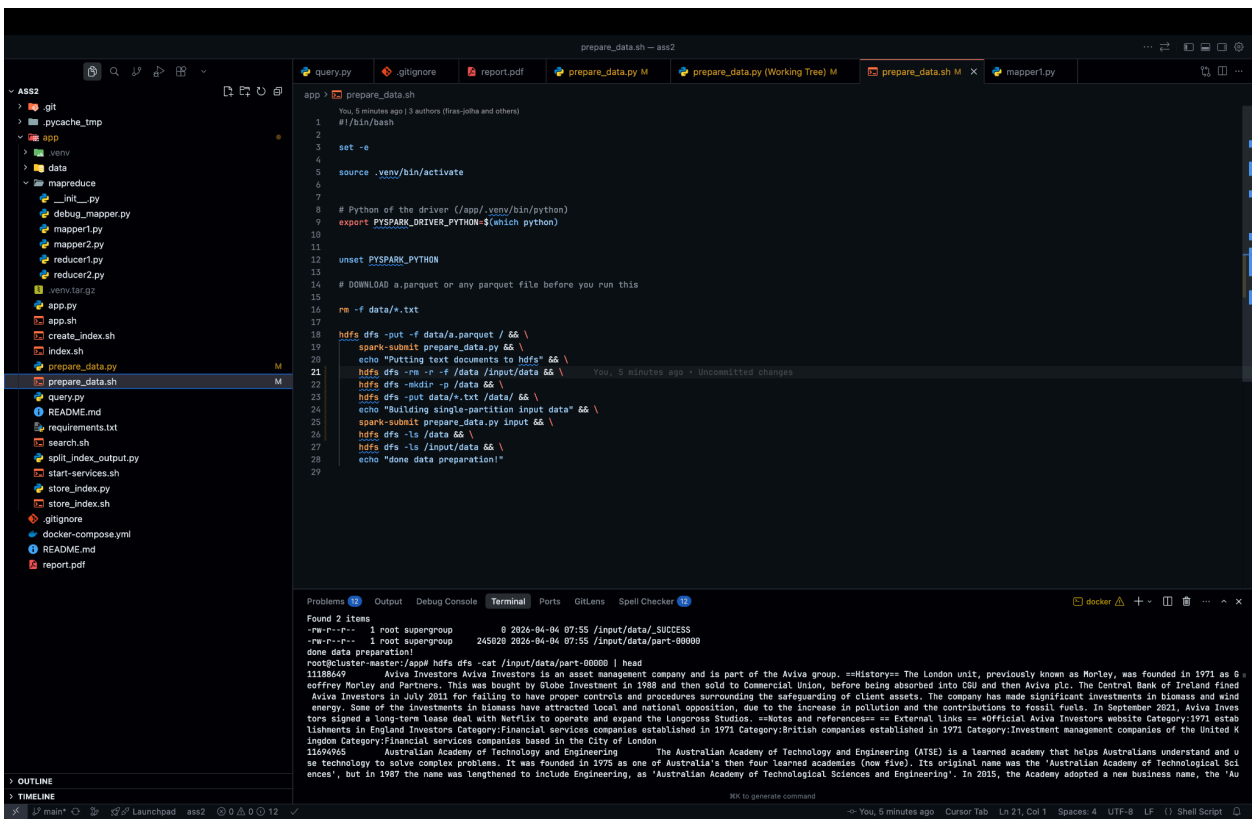
The serving layer is implemented by store_index.sh and store_index.py. store_index.sh validates that all required HDFS index artifacts exist, copies them into a local staging directory, waits until Cassandra is reachable, and then calls store_index.py. store_index.py creates the keyspace and four tables: postings_by_term, doc_stats, corpus_stats, and vocabulary. postings_by_term stores the inverted index keyed by term and doc_id. doc_stats stores title and document length per document. corpus_stats stores one global row with document_count, total_doc_length, and average_doc_length. vocabulary stores one row per term with its document frequency. Before loading, the tables are truncated so that the serving index is rebuilt from scratch in a consistent way.

The ranker is implemented by query.py and search.sh. query.py reads the query text from the command line or standard input, tokenizes it, connects to Cassandra, loads the corpus statistics, loads all document statistics, and retrieves postings and document frequencies only for the query terms. It then starts a Spark application, parallelizes the query-term payloads, computes BM25 scores for each matching document, sums the partial scores across all query terms with reduceByKey, and returns the top 10 highest-scoring documents. The BM25 implementation uses document frequency, term frequency, document length, average document length, and corpus size, with fixed hyperparameters k1 = 1.5 and b = 0.75. search.sh is the user-facing command. It validates the query, activates the local virtual environment for the driver, submits query.py to Spark on YARN, and configures the executors to use the system Python interpreter available on the worker containers.

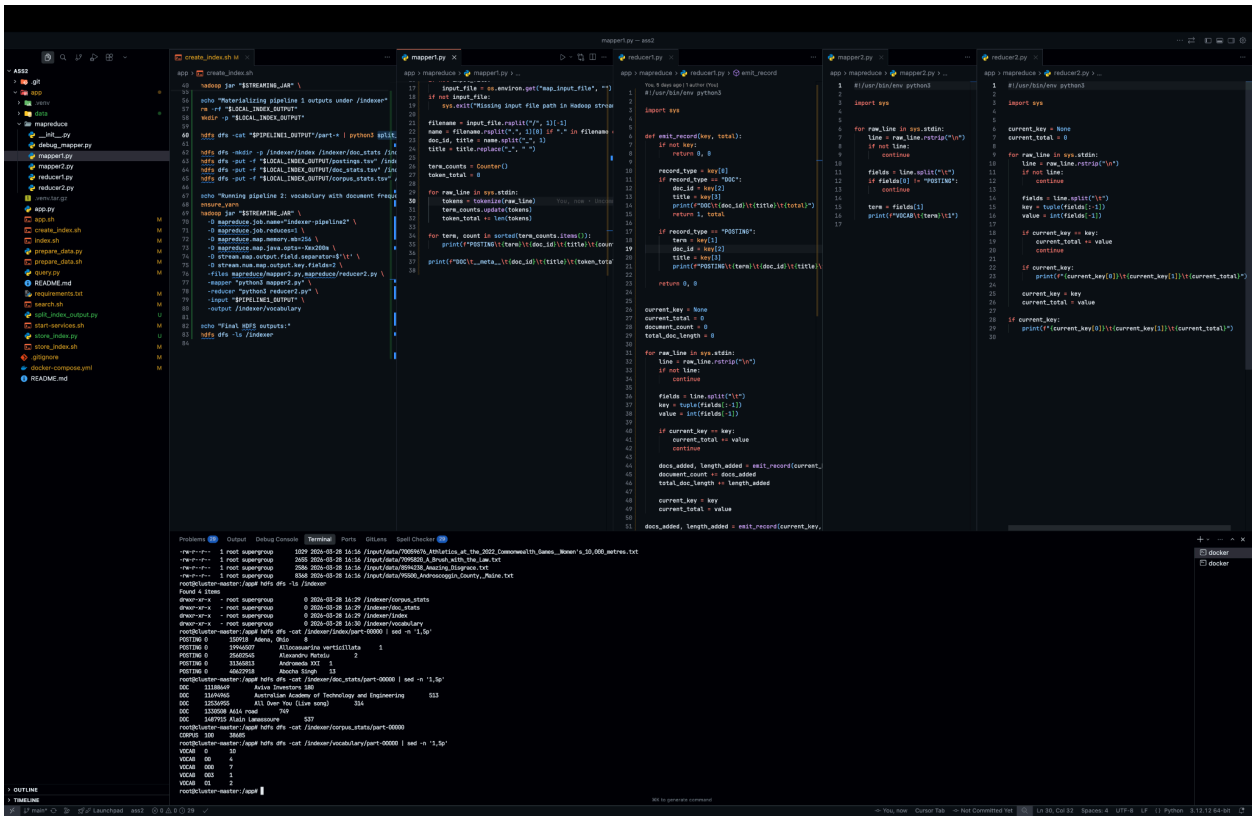
The required files in the assignment are therefore covered as follows. app.sh runs the full pipeline. start-services.sh starts the Hadoop services. prepare_data.py and prepare_data.sh create the raw documents and the prepared MapReduce input. mapper1.py and reducer1.py implement the first indexing pipeline. mapper2.py and reducer2.py implement the second indexing pipeline. create_index.sh builds the HDFS index. store_index.sh and store_index.py load the index into Cassandra. index.sh runs the full indexing flow. query.py performs BM25 ranking. search.sh runs the query on YARN. requirements.txt lists the Python dependencies. The final system keeps the architecture minimal: MapReduce is used only for offline index construction, Cassandra is used as the serving store, and Spark is used where distributed preparation and ranking are required.

Demonstration

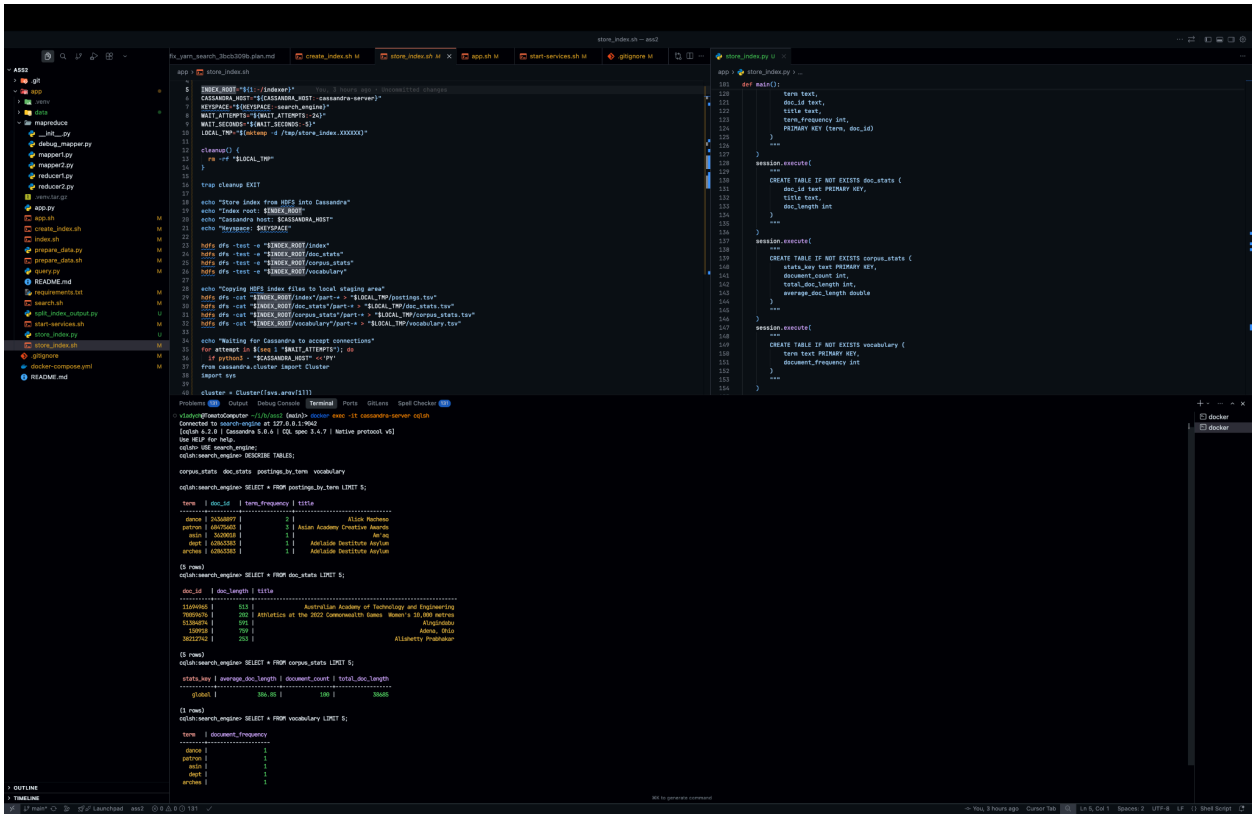
Proper work of prepare_data script: 100 documents (The assignment requires 100 documents for indexing in report instructions section and 1000 in description of the download pipeline. I am limited in resources hence I use 100 documents for putting into hdfs)



Mappers/Reducers scripts and results in HDFS

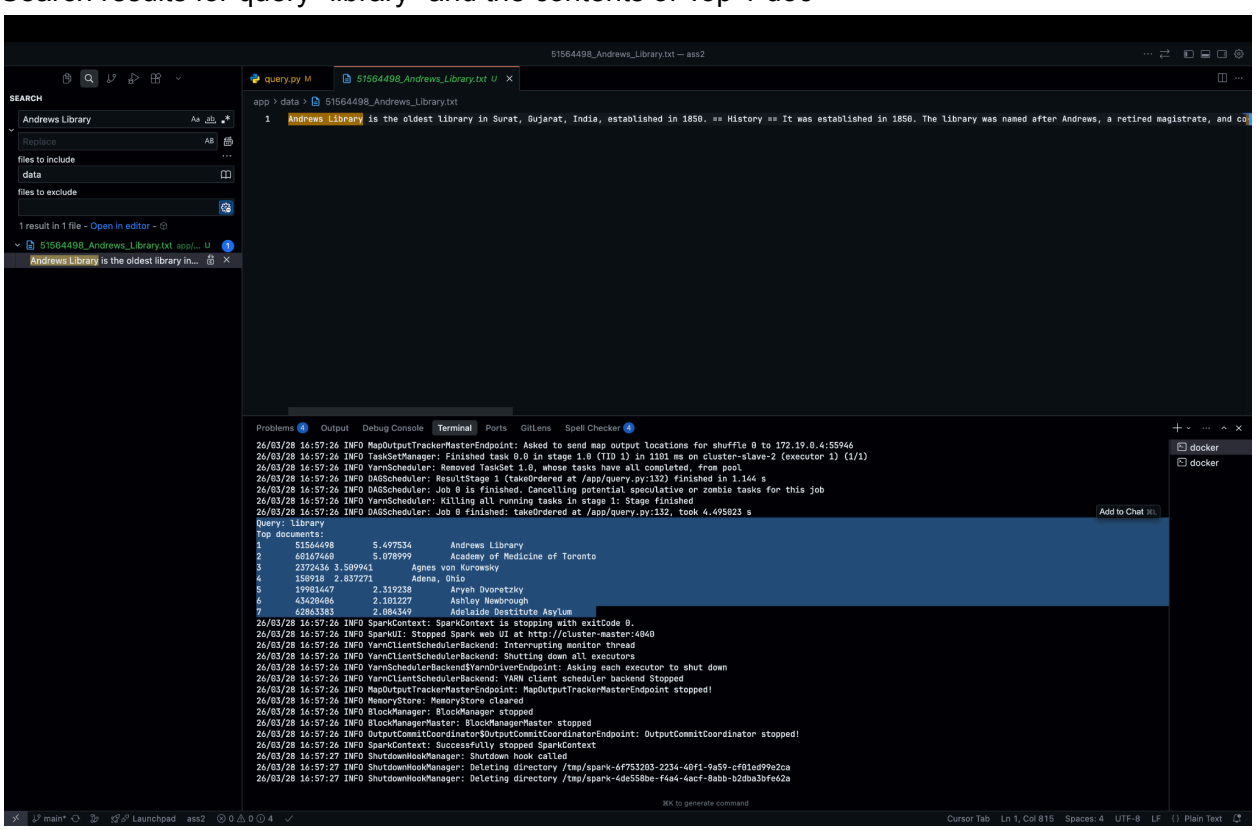
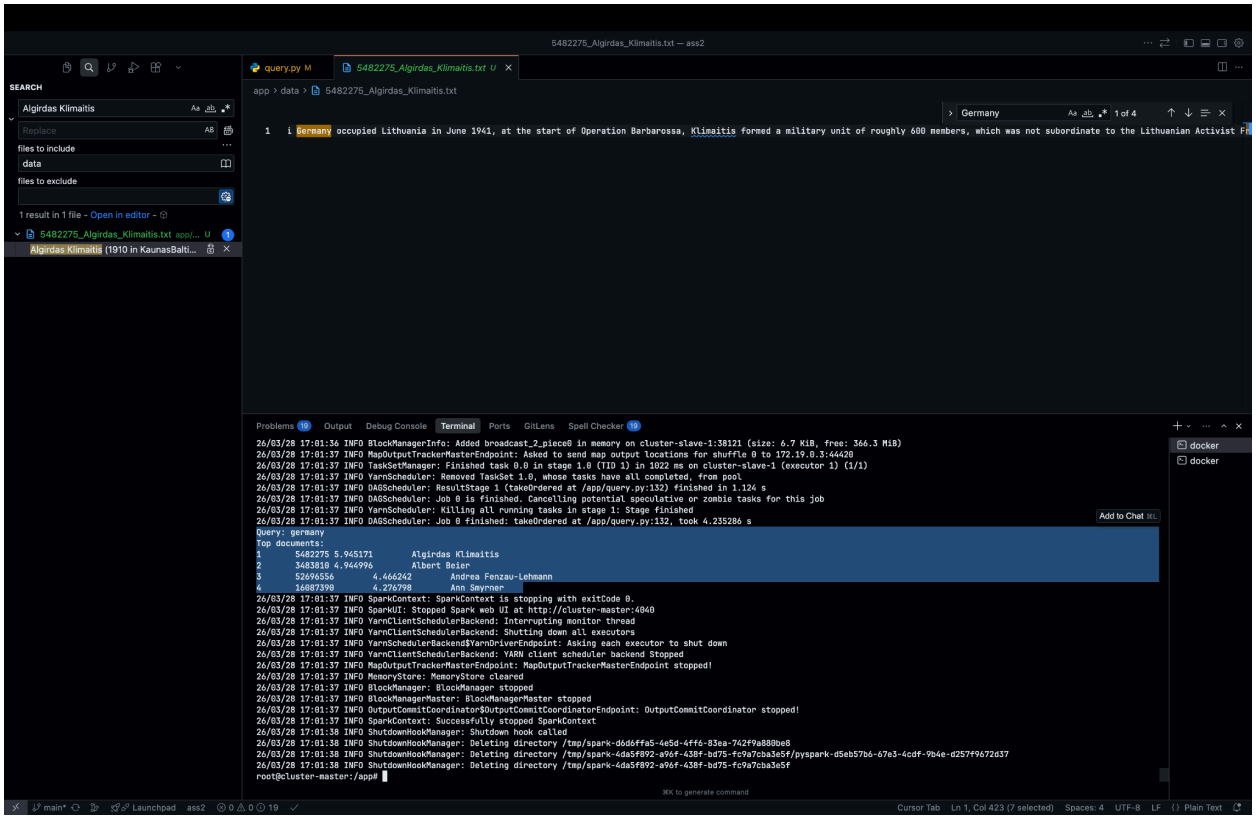


Cassandra transfer script and results in cassandra



Querying the index

Search results for query "library" and the contents of Top 1 doc



Search results for query "germany" and the contents of top 1 doc